

Guida al Progetto: Flashcards App

Un percorso di sviluppo passo dopo passo in C# e ADO.NET | The C# Academy

Benvenuto nella guida ufficiale strutturata per la realizzazione del progetto **Flashcards**. Questa roadmap ti accompagnerà dall'analisi iniziale del database fino all'implementazione dei requisiti logici e visuali dell'applicazione console, assicurandoti di strutturare il codice in modo professionale secondo i paradigmi della programmazione a oggetti (OOP).

Passo 1: Analisi dei Requisiti e Configurazione del Database

Prima di iniziare a scrivere codice C#, è essenziale definire l'architettura dei dati. Il progetto si basa su una struttura relazionale per tracciare i mazzi di carte (Stacks), le singole domande/risposte (Flashcards) e lo storico delle sessioni di studio (Study Sessions).

- **Stack (Mazzo):** Identificato da un nome univoco (es. "C# Basics", "Spagnolo A1").
- **Flashcard (Carta):** Collegata a uno specifico Stack. Contiene il testo frontale (domanda) e il testo sul retro (risposta).
- **StudySession (Sessione di Studio):** Tiene traccia della data/ora, dello Stack studiato e del punteggio ottenuto.

È di fondamentale importanza configurare i vincoli di integrità referenziale, utilizzando la clausola **ON DELETE CASCADE**. Questo garantisce che, nel momento in cui un utente elimina un intero mazzo, tutte le carte associate e lo storico vengano rimossi automaticamente in cascata dal database, prevenendo record orfani.

Passo 2: Creazione del Progetto C# ed Architettura dei Livelli

Inizializza una nuova applicazione console .NET. Per mantenere il codice scalabile e pulito, è consigliato evitare di inserire tutta la logica all'interno di un unico file. Suddividi il progetto in quattro cartelle principali:

1. **Models:** Classi pure che rappresentano le entità del database (`Stack.cs` , `Flashcard.cs` , `StudySession.cs`).
2. **DataAccess:** Classi responsabili dell'esecuzione delle query SQL e della gestione delle connessioni.
3. **Controllers:** Classi che contengono la logica di business e coordinano il flusso tra dati e interfaccia utente.
4. **Views / UI:** Gestione esclusiva dell'input dell'utente, dei menu a schermo e della formattazione dell'output.

Suggerimento di stile C#: Abilita la funzionalità degli *implicit usings* nel file di progetto (.csproj) per importare staticamente il tipo `System.Console`, riducendo la verbosità del codice durante la scrittura dei comandi di input/output.

Passo 3: Sviluppo del Livello di Accesso ai Dati (SQL & ADO.NET)

Configura l'infrastruttura per permettere all'applicazione di interagire con il database SQL Server (o SQLite). Puoi utilizzare ADO.NET nativo (`SqlCommand` , `SqlConnection`) oppure mappare le entità in modo più rapido tramite il micro-ORM **Dapper**.

Il tuo livello di Data Access deve esporre metodi robusti per eseguire le seguenti operazioni CRUD:

- Estrazione e inserimento di nuovi mazzi (Stacks).
- Lettura delle Flashcards filtrate per lo Stack selezionato.
- Inserimento, modifica ed eliminazione delle singole Flashcards.
- Salvataggio e recupero dello storico delle sessioni (Study Sessions).

Passo 4: Implementazione dei DTO (Data Transfer Objects)

Un requisito esplicito di questo progetto consiste nel nascondere all'utente finale le chiavi primarie reali del database (gli ID autoincrementali come 1, 47, 108). Mostrare ID con "buchi" dovuti a passate eliminazioni rende l'interfaccia confusa.

Per risolvere questo problema, implementa il pattern DTO:

- Crea una classe `FlashcardDto` che, al posto dell'ID reale, contenga una proprietà denominata `DisplayId` .
- Quando recuperi le carte dal database, racchiudile in una lista e mappa a runtime il `DisplayId` in modo sequenziale lineare, partendo da **1** fino a **N** (dove **N** è il numero totale di carte presenti nel mazzo visualizzato).
- L'utente interagirà solo con il `DisplayId` . Il controller si occuperà poi di convertire questo valore nell'ID reale del database prima di inviare i comandi di modifica o rimozione.

Passo 5: Costruzione del Menu Principale e dell'Interfaccia Utente

Sviluppa l'interfaccia utente basandoti su un ciclo continuo (`while`) posizionato all'interno del modulo UI. Per rendere l'applicazione professionale, puoi integrare la libreria NuGet **Spectre.Console**, utile per generare tabelle, menu di selezione interattivi e titoli colorati.

Il menu di navigazione principale deve presentare le seguenti opzioni:

1. **Gestione Mazzi (Stacks):** Sottomenu per creare, visualizzare o eliminare i mazzi.
2. **Gestione Flashcards:** Selezione preventiva dello Stack di interesse, seguita dalle opzioni per visualizzare le carte (tramite DTO), aggiungerne di nuove, modificarle o eliminarle.
3. **Sessione di Studio (Study):** Avvio del modulo di apprendimento interattivo.
4. **Storico Sessioni:** Visualizzazione ordinata dei punteggi passati.
5. **Uscita:** Chiusura sicura dell'applicazione.

Passo 6: Logica della Sessione di Studio e Calcolo del Punteggio

La sessione di studio rappresenta il motore operativo dell'applicazione. Il flusso deve seguire rigorosamente questi step:

1. L'utente seleziona lo Stack che desidera ripassare.
2. Il sistema estrae tutte le Flashcards associate a quel mazzo e inizializza la variabile del punteggio attuale: $S = 0$.
3. Per ciascuna carta estratta, il programma mostra a schermo esclusivamente il testo contenuto nella proprietà `Front` (la domanda) e attende l'input testuale dell'utente.
4. Ricevuto l'input, il sistema esegue una comparazione logica con la proprietà `Back` (la risposta corretta). La verifica deve essere *case-insensitive* e ripulita da spazi bianchi superflui all'inizio o alla fine, utilizzando i metodi `.Trim().ToLower()`.
5. Se la risposta coincide, l'applicazione incrementa il punteggio ($S = S + 1$) e mostra un messaggio di conferma. In caso contrario, mostra la risposta corretta.
6. Al termine del mazzo, il sistema calcola la percentuale di successo ed inserisce una nuova riga nella tabella `StudySessions`, registrando l'ID del mazzo, il timestamp corrente (`DateTime.Now`) e il punteggio finale conseguito.

Passo 7: Validazione dei Dati e Gestione degli Errori

Un'applicazione solida deve prevenire i crash improvvisi derivanti da inserimenti imprevisti da parte dell'utente. Implementa i seguenti controlli di sicurezza:

- **Validazione dei Numeri:** Utilizza sempre il metodo `int.TryParse()` per l'acquisizione degli ID o delle scelte numeriche nei menu. Se l'utente digita caratteri testuali anziché numeri, l'applicazione deve mostrare un messaggio d'errore e richiedere l'input senza interrompersi.
- **Verifica Stringhe Vuote:** Impedisci la creazione di mazzi, domande o risposte prive di testo, verificando i dati immessi con il metodo `string.IsNullOrEmpty()`.
- **Controllo dei Mazzi Vuoti:** Se l'utente avvia una sessione di studio su un mazzo privo di carte, il sistema deve intercettare lo stato prima di avviare il ciclo, informando l'utente con un messaggio dedicato ed invitandolo ad aggiungere delle Flashcards prima di procedere.